

Locality-sensitive Hashing II

COMPSCI 753

Zijian Huang

The University of Auckland

RECAP: MinHashing

Permutation π

4
2
7
1
6
3
5

	A	B	C	D
1	1	1	1	0
2	1	1	0	1
3	0	1	0	1
4	0	0	0	1
5	1	0	0	1
6	1	1	1	0
7	1	0	1	0
Documents				

Shingles

After
Permutation

	A	B	C	D
1	0	0	0	1
2	1	1	0	1
3	1	1	1	0
4	1	1	1	0
5	1	0	1	0
6	1	0	0	1
7	0	1	0	1
Documents				

Shingles

RECAP: MinHashing

Permutation π

4	3	2
2	4	3
7	7	7
1	2	6
6	6	1
3	1	5
5	5	4

	A	B	C	D
1	1	1	1	0
2	1	1	0	1
3	0	1	0	1
4	0	0	0	1
5	1	0	0	1
6	1	1	1	0
7	1	0	1	0

Shingles



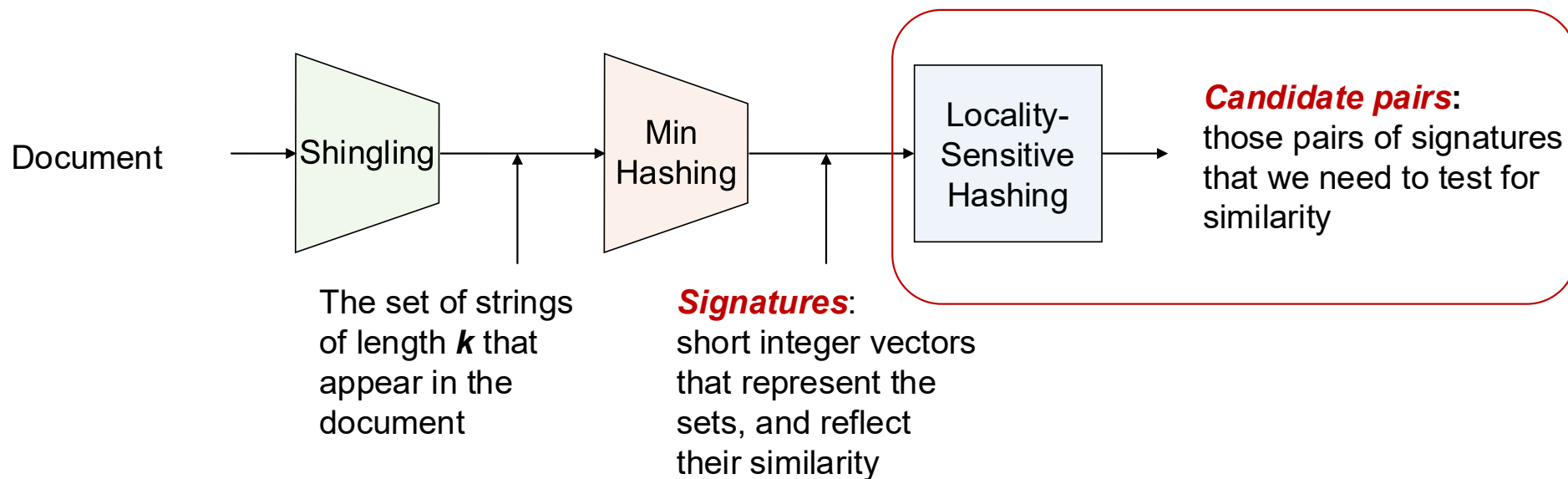
Signature matrix M

	A	B	C	D
	2	2	3	1
	1	1	1	2
	1	2	2	1

Similarity $J(S, T)$	(A,B)	(C,D)
Binary Vector	3/6=0.5	0/7=0
Signature	2/3=0.67	0/3=0

Documents

Framework Overview



LSH: Motivation

- $n = 1$ million documents
- A naïve solution needs to compute $n(n - 1) / 2 \sim 5 \times 10^{11}$ Jaccard similarities!!
- A standard PC computes 106 Jaccard similarities/sec, then we need approximate 5 days
- Nowadays, the internet has $n \sim 4.5$ billion pages (OMG!)

LSH: Problem Definition

- Input: The MinHash signature matrix M

Signature Matrix M

	A	B	C	D
x_1	2	2	3	1
x_2	1	1	1	2
x_3	1	2	2	1

$r = 3$

- Goal: Find all pairs of documents with Jaccard similarity $s \geq t = 0.8$
- Idea of LSH
 - The higher Jaccard similarity, the larger fractions of identical hash values
 - A pair of signatures is a **candidate** if their hash values are the same on at least r fractions
- Problem: Choose r to minimize the number of
 - False negatives** (similar pairs with $J \geq 0.8$ but not candidates) and
 - False positives** (dissimilar pairs with $J < 0.8$ but candidates)

Naïve Solution and Problem Analysis

- $r = 1$:
 - $J(S, T) = 0.8$ (truly similar, should be a candidate pair)
 - Probability that (S, T) is a candidate: 0.8
 - Probability that (S, T) is not a candidate: $1 - 0.8 = 0.2$ ← False Negative
 - $J(S', T') = 0.5$ (truly dissimilar, should NOT be a candidate pair)
 - Probability that (S', T') is a candidate: 0.5 ← False Positive
 - Probability that (S', T') is not a candidate: $1 - 0.5 = 0.5$
- $r = 5$:
 - $J(S, T) = 0.8$ (truly similar, should be a candidate pair)
 - Probability that (S, T) is a candidate: $0.8^5 = 0.328$
 - Probability that (S, T) is not a candidate: $1 - 0.8^5 = 0.672$ ← False Negative
 - $J(S', T') = 0.5$ (truly dissimilar, should NOT be a candidate pair)
 - Probability that (S', T') is a candidate: $0.5^5 = 0.031$ ← False Positive
 - Probability that (S', T') is not a candidate: $1 - 0.5^5 = 0.969$

LSH: Partition M into b Bands

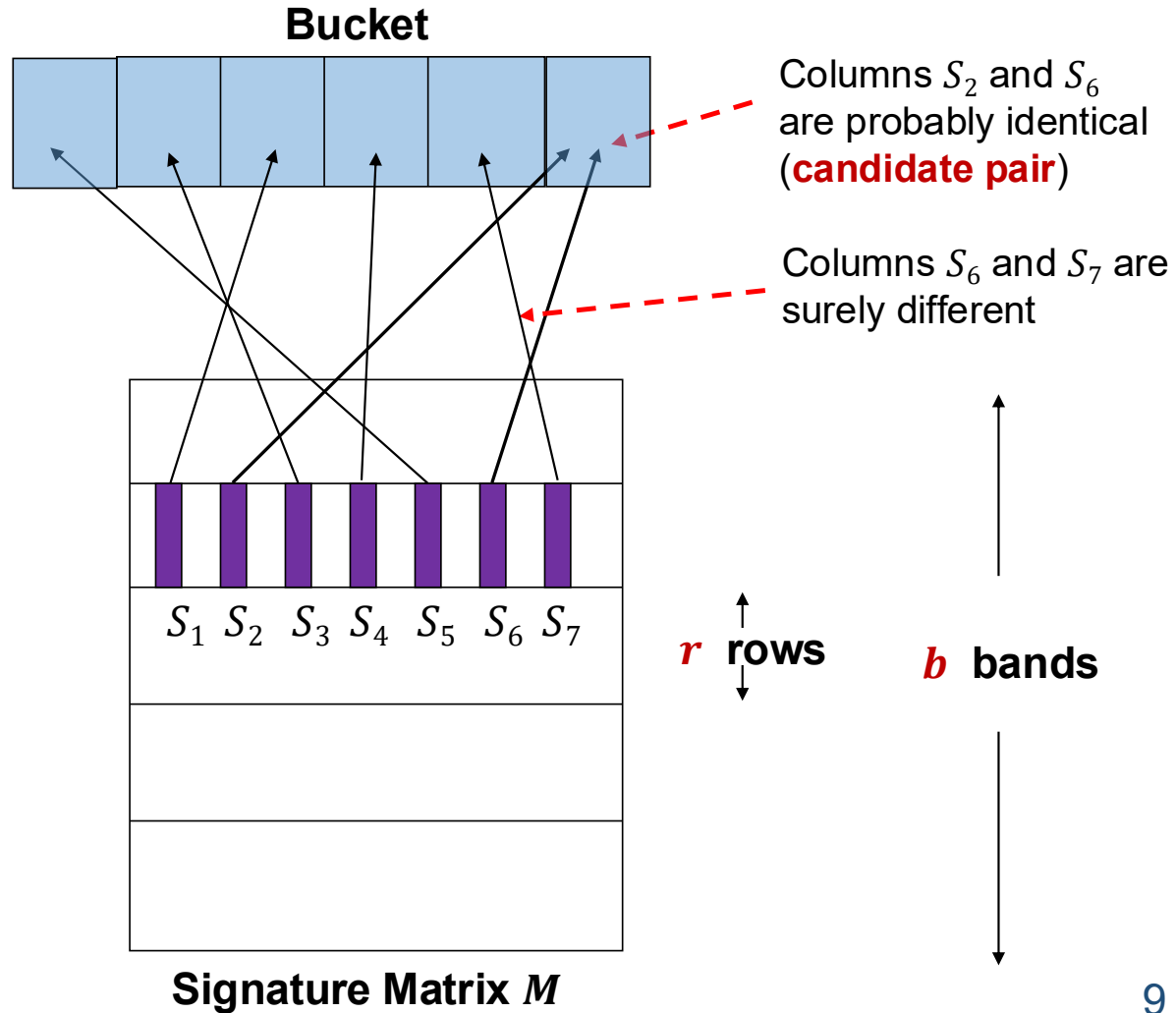
Signature Matrix M

Partition signature matrix M into $b = 4$ bands $\rightarrow$

band 1	...	1 0 0 0 2 3 2 1 2 2 0 1 3 1 1	...	$\leftarrow r = 3$ rows per hand
band 2		$\leftarrow 1$ signature		
band 3				
band 4				

LSH: Partition M into b Bands

- M contains b bands, each band has r rows
- For each band, use **general hash function** to hash its r rows of each column into a hash table with very large k buckets
 - The collision probability of two different rows is super tiny
- For each band, **candidates** are column pairs hashing to the same bucket
 - Candidate pairs might be on the same bucket for more than 1 band
- **Tune b and r** to return most truly similar pairs but few truly dissimilar pairs



Example: $r = 3, b = 1$

Level 1: Permutation Simulation for Signature Generation ($r = 3, p, n$):

1. $h_{a_{1,1}, b_{1,1}}: \text{num_features } n \rightarrow \text{num_features } n$
2. $h_{a_{1,2}, b_{1,2}}: \text{num_features } n \rightarrow \text{num_features } n$
3. $h_{a_{1,3}, b_{1,3}}: \text{num_features } n \rightarrow \text{num_features } n$

$$h_{a,b}(x) = ((ax + b) \bmod p) \bmod n$$

$$0 < a < p, 0 < b < p$$

Level 2: Signature Hashing for Hash Table Construction ($b = 1, r = 3, p, m$):

1. $g_{c_{1,0}, c_{1,1}, c_{1,2}, c_{1,3}}: r\text{-dim signatures} \rightarrow \text{num_buckets } m$

$$g_{c_{1,0}, c_{1,1}, c_{1,2}, c_{1,3}}([x_1, x_2, x_3])$$

$$= ((c_{1,0} + c_{1,1}x_{1,1} + c_{1,2}x_{1,2} + c_{1,3}x_{1,3}) \bmod p) \bmod m$$

$$0 < c_{1,1}, c_{1,2}, c_{1,3} < p, 0 < c_{1,0} < p$$

Signature matrix M_1

	A	B	C	D
x_1	2	2	3	1
x_2	1	1	1	2
x_3	1	2	2	1

$r = 3$

Hash Table ($b = 1$)

m buckets					
B	A		C		D

$b = 1$

Example: $r = 3$, $b = i$ ($i > 1$)

Level 1: Permutation Simulation for Signature Generation ($r = 3, p, n$):

1. $h_{a_{i,1}, b_{i,1}}: \text{num_features } n \rightarrow \text{num_features } n$
2. $h_{a_{i,2}, b_{i,2}}: \text{num_features } n \rightarrow \text{num_features } n$
3. $h_{a_{i,3}, b_{i,3}}: \text{num_features } n \rightarrow \text{num_features } n$

$$h_{a_{i,j}, b_{i,j}}(x) = ((a_{i,j}x + b_{i,j}) \bmod p) \bmod n$$

$$0 < a_{i,j} < p, 0 < b_{i,j} < p$$

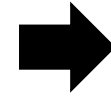
Level 2: Signature Hashing for Hash Table Construction ($b = i, r = 3, p, m$):

1. $g_{c_{i,0}, c_{i,1}, c_{i,2}, c_{i,3}}: r\text{-dim signatures} \rightarrow \text{num_buckets } m$

$$g_{c_{i,0}, c_{i,1}, c_{i,2}, c_{i,3}}([x_{i,1}, x_{i,2}, x_{i,3}])$$

$$= ((c_{i,0} + c_{i,1}x_{i,1} + c_{i,2}x_{i,2} + c_{i,3}x_{i,3}) \bmod p) \bmod m$$

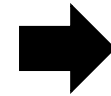
$$0 < c_{i,1}, c_{i,2}, c_{i,3} < p, 0 < c_{i,0} < p$$



$r = 3$

Signature matrix M_i

	A	B	C	D
x_1	1	2	2	1
x_2	2	2	3	1
x_3	1	1	1	2



$b = 1$

	m buckets				
$b = 1$	B	A		C	D
	...				
$b = i$	B	C		A	D

LSH: Partition M into b Bands

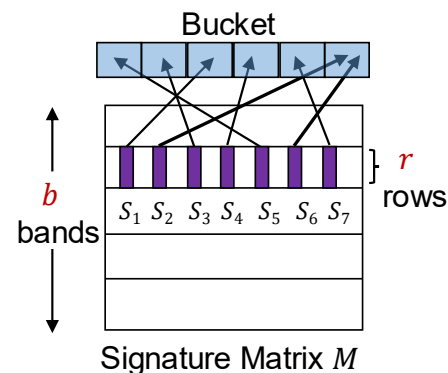
- Assumptions
 - The number of buckets is sufficiently large so that two **different columns** will hash into **different buckets**
 - “**same bucket**” means “**identical in that band**”
 - The assumption will simplify analysis, not for correctness of algorithms

Example: Analysis for $J(S, T) = 0.8$

- Input: Divide matrix M of $d = 100$ rows into $b = 20, r = 5$
- Goal: (S, T) is a candidate, then S and T should be hashed into the same bucket for at least 1 band

- Analysis

- Probability that (S, T) identical in 1 band: $0.8^5 = 0.328$
- Probability that (S, T) not identical in 1 band: $1 - 0.8^5 = 0.672$
- Probability that (S, T) not identical in all 20 bands: $(1 - 0.8^5)^{20} = 0.00035$
- Probability that (S, T) **identical in at least 1 band**: $1 - 0.00035 = 0.99965$



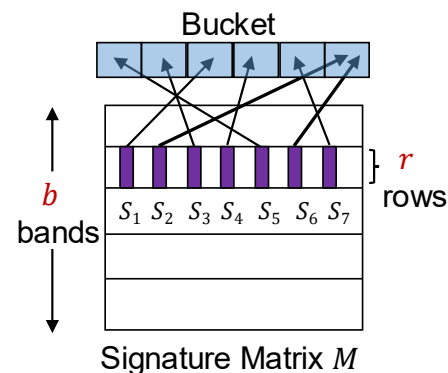
- Conclusion
 - About **0.00035** of the 80%-similar column pair are not hashed into at least one bucket → **false negatives**
 - We can find **0.99965 truly similar pairs of documents**

Example: Analysis for $J(S, T) = 0.3$

- Input: Divide matrix M of $d = 100$ rows into $b = 20, r = 5$
- Goal: (S, T) is not a candidate, then S and T are not hashed into the same bucket for all bands

- Analysis

- Probability that (S, T) identical in 1 band: $0.3^5 = 0.00243$
- Probability that (S, T) not identical in 1 band: $1 - 0.3^5 = 0.99757$
- Probability that (S, T) not identical in all 20 bands: $(1 - 0.3^5)^{20} = 0.9525$
- Probability that (S, T) **identical in at least 1 band**: $1 - 0.9525 = 0.0475$



- Conclusion
 - Approximately **0.0475** of the 30%-similar column pair are candidate pair $\rightarrow$ **false positives**
 - We can compute their actual Jaccard similarity and choose not to report them

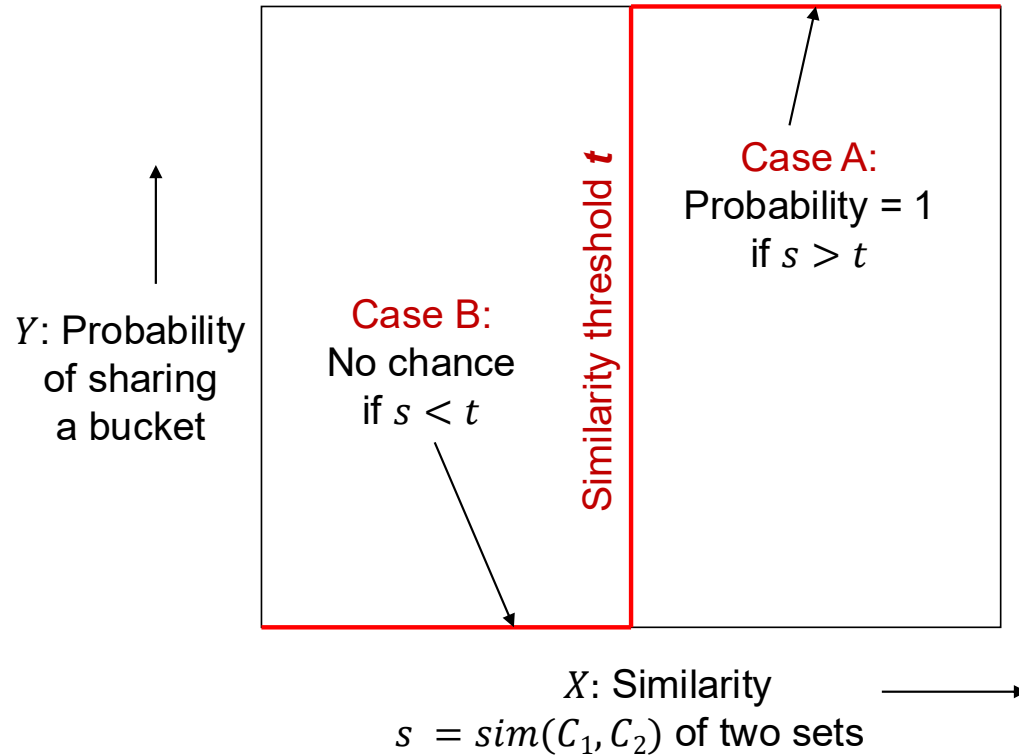
LSH: Trade-Off

- Tune parameter settings to balance the false negatives/positives
 - The number of MinHash permutation d to construct the signature matrix M
 - The number of bands b
 - The number of rows r per band
- Example: given $b = 15, r = 5$ rows
 - What is the probability 80%-similar pair is not a candidate?
 - What is the probability 30%-similar pair is a candidate?
 - False negatives increase, false positives decrease

LSH: General Analysis for $J(S, T) = s$

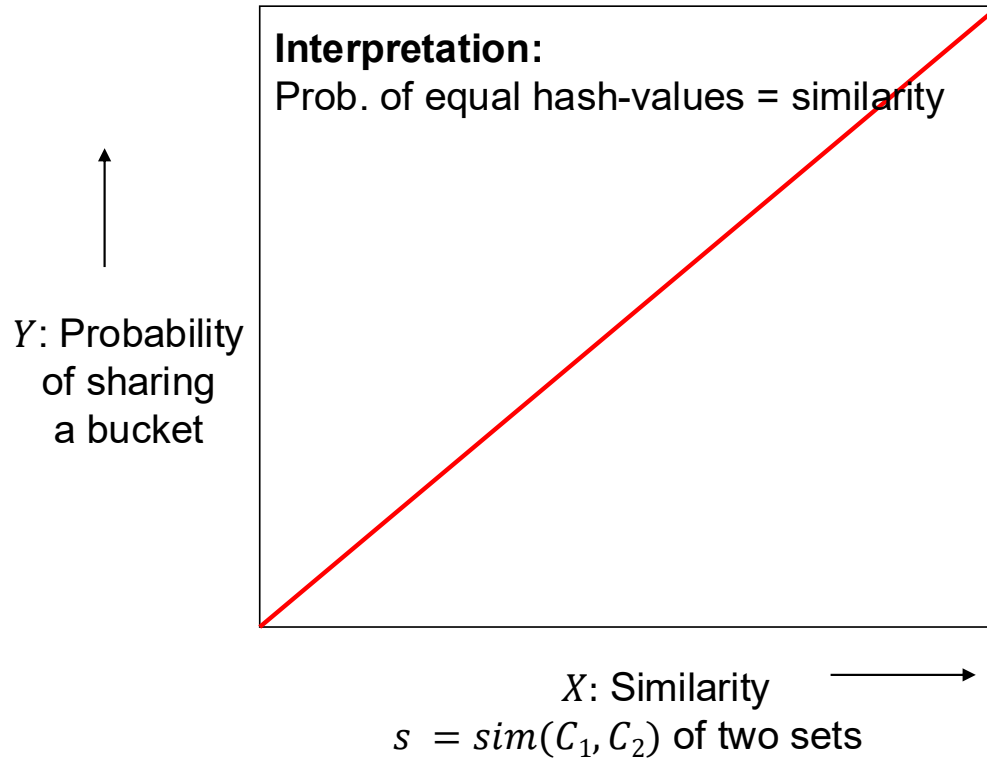
- Parameters: b bands and r rows per band
- Analysis
 - Probability that (S, T) identical in 1 band: s^r
 - Probability that (S, T) not identical in 1 band: $1 - s^r$
 - Probability that (S, T) not identical in all b bands: $(1 - s^r)^b$
 - Probability that (S, T) is not a candidate: $(1 - s^r)^b$
 - Probability that (S, T) is a candidate: $1 - (1 - s^r)^b$

LSH: Ideal Case



- s : similarity of two sets
- t : similarity threshold
- X : pair similarity
- Y : prob. of sharing a bucket

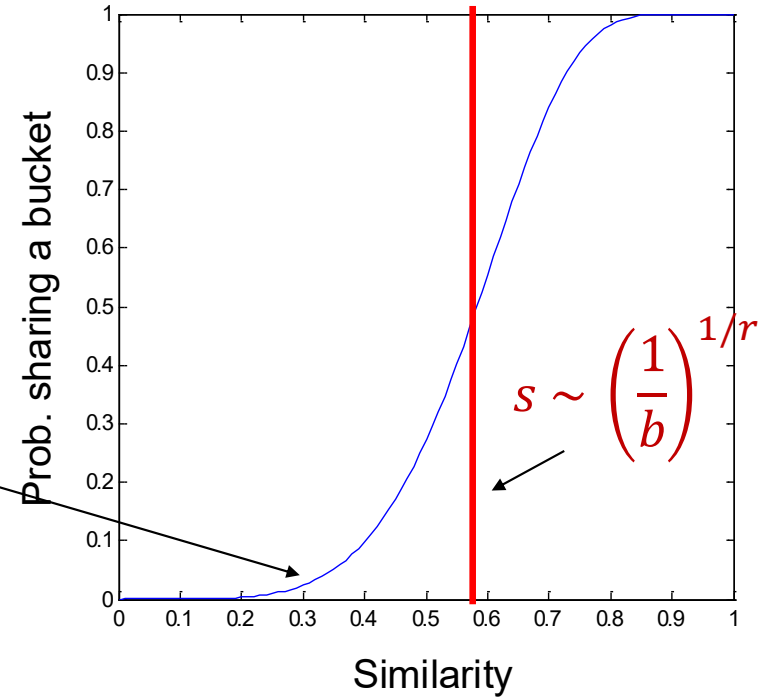
LSH: ($b = 1, r = 1$)



- s : similarity of two sets
- t : similarity threshold
- X : pair similarity
- Y : prob. of sharing a bucket

LSH: General b and r

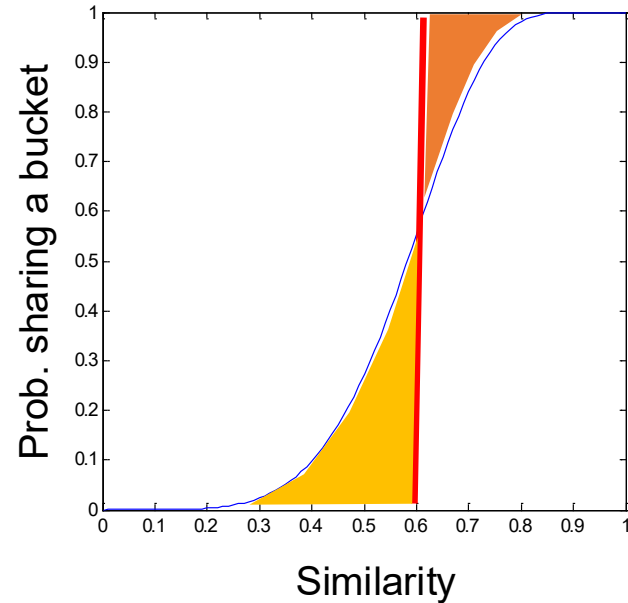
- Probability that (S, T) identical in 1 band: s^r
- Probability that (S, T) not identical in 1 band: $1 - s^r$
- Probability that (S, T) not identical in all b bands: $(1 - s^r)^b$
- Probability that (S, T) is not a candidate: $(1 - s^r)^b$
- Probability that (S, T) is a candidate: $1 - (1 - s^r)^b$



LSH: General b and r

- Tune b and r to get the best S-curve
- Example: $b = 10, r = 5, d = 50$

Orange area: False Negative rate
Yellow area: False Positive rate



Visualization of S-Curve: <https://www.desmos.com/calculator/lzzvfjiujn>

LSH: Overview

1. Tune b and r to get the best S-curve
 - **Minimize false negatives**: get almost all pairs of truly similar signatures
 - **Minimize false positives**: eliminate most truly dissimilar pairs
1. Validate candidate pairs truly share similar signatures by Jaccard similarity estimation in main memory
2. Optional: Take another scan through data, compute the actual Jaccard similarity of candidate pairs to return the result

Summary

- **Shingling:** Convert documents to sets
 - We used hashing to assign each shingle an integer ID.
- **Min-Hashing:** Convert large sets to short signatures, while preserving Jaccard similarity
 - We used similarity preserving hashing to generate signatures with property
$$\Pr_{\pi}[h_{\pi}(S) = h_{\pi}(T)] = J(S, T).$$
 - We used hashing to get around generating random permutations efficiently
- **Locality-Sensitive Hashing:** Focus on pairs of signatures likely to be from similar documents.
 - Use hashing to find candidate pairs of Jaccard similarity $\geq t$.

Resources

- Visualization of S-Curve: <https://www.desmos.com/calculator/lzzvfjiujn>
- **[Book]** Algorithm Design by Jon Kleinberg and Éva Tardos. Chapter 13
 - <https://www.cs.princeton.edu/~wayne/kleinberg-tardos/>
- **[Book]** Mining of Massive Datasets by Jure Leskovec, Anand Rajaraman, and Jeff Ullman. Chapter 3
 - <http://mmds.org>